



AI with Rav

Learn AI the way you actually think.



DAY 2 of 30

PYTHON

Variables & Data Types — labelled boxes for your data

WHAT YOU'LL LEARN TODAY

- › What a variable really is
- › The 4 everyday data types
- › How to re-fill a variable
- › Naming rules (and what to avoid)
- › Checking a type with `type()`



Variables & Data Types — labelled boxes for your data

In One Line: A variable is just a labelled box that holds a value. You put something in ("age = 30"), give it a name ("age"), and use that name whenever you need what's inside. That's the building block of every program.

Start here: your program needs a memory

Yesterday you printed text. But a real program needs to **remember** things — a user's name, a price, a score — and use them later. For that, we use **variables**. A variable is the simplest, most important idea in all of programming, and it's beautifully easy.

Think of a variable as a **labelled box**. You put a value inside the box and stick a name on it. Later, whenever you say the name, Python hands you what's inside.

A variable is a labelled box

A variable = a labelled box that holds a value



A yellow box holds the value 30, with the label "age" stuck on top. The code `age = 30` creates it: "age" is the label, 30 is what goes inside the box.

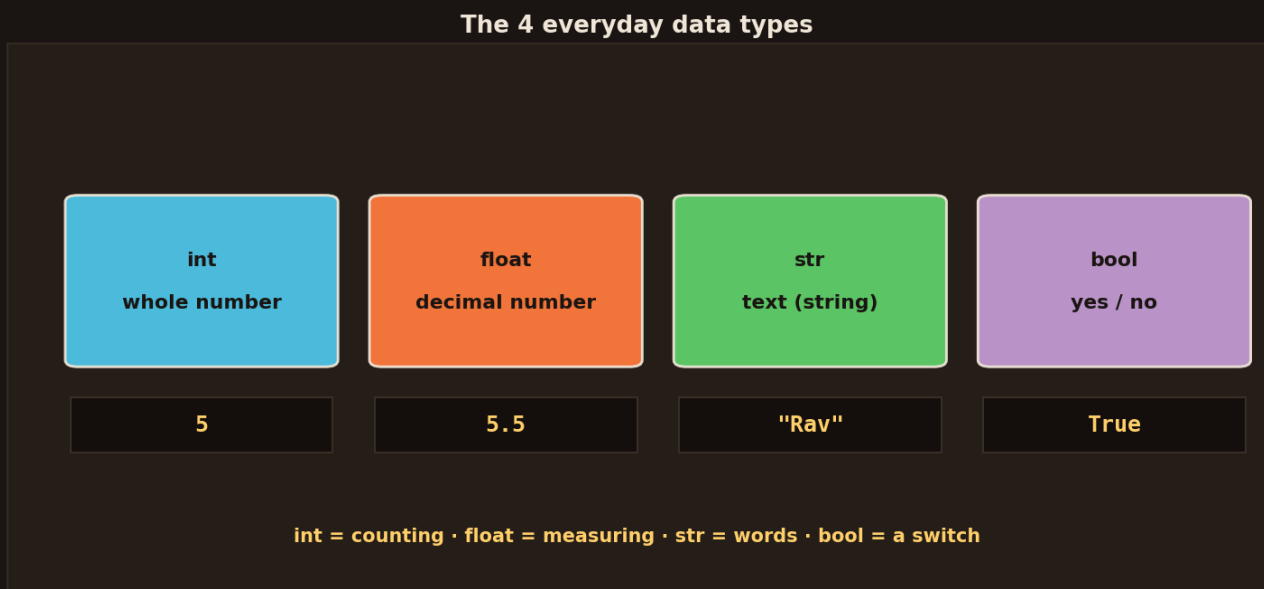
```
age = 30
print(age)      # -> 30
```

- ``age`` → the **name** (the label on the box).
- ``=`` → means "put this value into this box." (It's NOT "equals" like maths — it means "store.")
- ``30`` → the **value** that goes inside.



(If you already code: no need to declare a type — Python figures out it's a number by itself. Just `name = value`.) Now, whenever you write `age`, Python replaces it with `30`. That's the whole idea — a name that stands for a stored value.

The 4 everyday data types



The four basic types: int (a whole number like 5), float (a decimal like 5.5), str (text in quotes like "Rav"), and bool (a yes/no value, True or False).

Boxes can hold different **kinds** of things. Python's four everyday types:

- **int** → a **whole number**, like `5` or `100`. (For counting.)
- **float** → a **decimal number**, like `5.5` or `9.99`. (For measuring.)
- **str** → **text** (a "string"), always in quotes, like `"Rav"`. (For words.)
- **bool** → a **yes/no** value: either `True` or `False`. (A simple switch.)

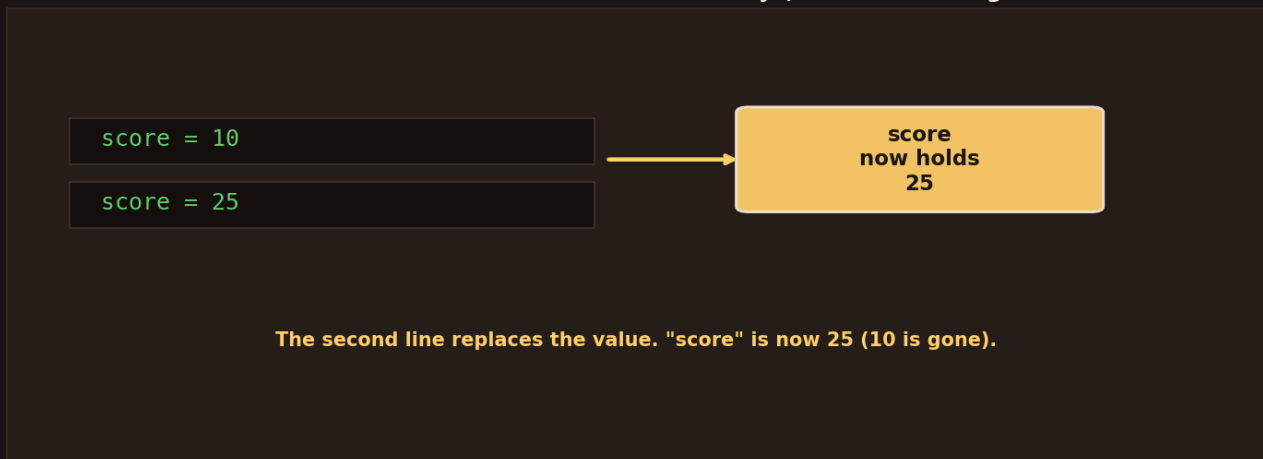
```
age    = 30          # int
price  = 9.99        # float
name   = "Rav"       # str
is_ready = True      # bool
```

Remember them by their job: **int = counting, float = measuring, str = words, bool = a switch**. You don't tell Python which type to use — it looks at the value and knows. `5` is an int, `5.5` is a float, `"5"` (with quotes) is a str. That last one surprises beginners: quotes make it text, not a number!



A box can be re-filled

A variable can be re-filled — the label stays, the value changes



The second line replaces the value. "score" is now 25 (10 is gone).

Writing `score = 10` then `score = 25` replaces the value. The label "score" stays, but now it holds 25 — the old 10 is gone.

Variables aren't fixed — you can put a new value in the same box anytime:

```
score = 10
print(score)    # -> 10

score = 25      # the box is re-filled
print(score)    # -> 25 (the old 10 is gone)
```

The second line doesn't create a new box — it **replaces** what's inside the old one. The label `score` stays the same; only the value changes. This is how programs track things that change: a running total, a game score, a countdown.

Naming your boxes — the rules

Good variable names

- ✓ `user_age`
- ✓ `total_price`
- ✓ `is_ready`
- ✓ `first_name`

clear · lowercase · words joined by _

Names to avoid

- ✗ `x` (too vague)
- ✗ `2fast` (can't start with a number)
- ✗ `my age` (no spaces allowed)
- ✗ `print` (a Python word)



Good names: `user_age`, `total_price`, `is_ready` — clear, lowercase, words joined by underscores. Names to avoid: `x` (too vague), `2fast` (can't start with a number), `"my age"` (no spaces), `print` (already a Python word).

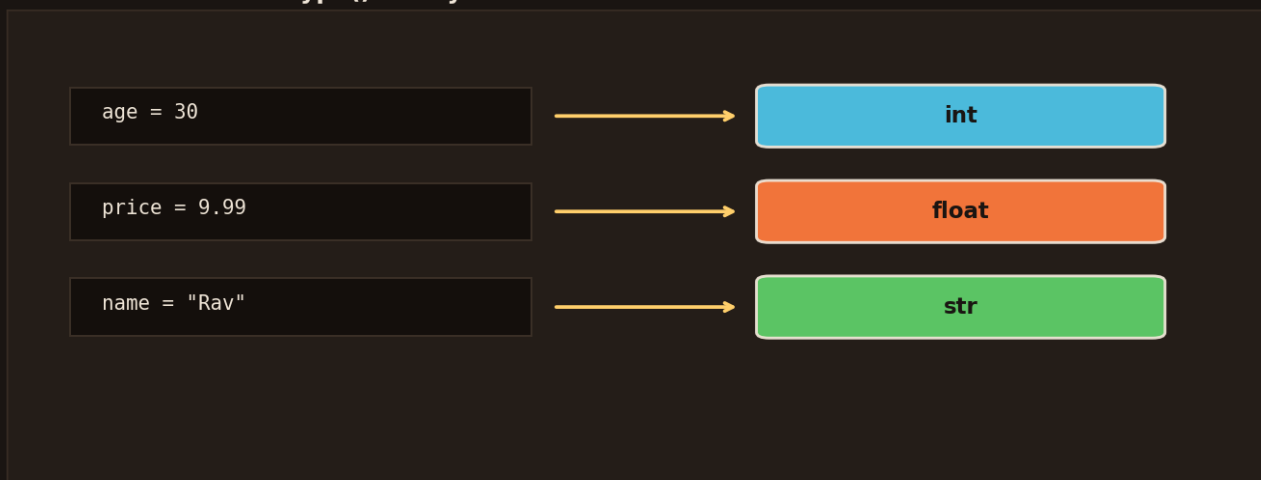
Good names make code readable. A few simple rules:

- **Do:** use clear, lowercase names, joining words with an underscore → ``user_age``, ``total_price``, ``is_ready``.
- **Don't:** start with a number (``2fast`` ❌), use spaces (``my age`` ❌), or reuse a Python word like ``print`` ❌.
- Avoid single vague letters like ``x`` — ``total_price`` tells the reader what it holds.

The habit that separates good coders: **name things clearly**. ``d = 30`` means nothing in six months; ``days_left = 30`` explains itself. Code is read far more often than it's written — write names your future self will thank you for.

Checking a type with `type()`

`type()` tells you what kind of value a variable holds



Calling `type(age)` on `age = 30` returns `int`; `type(price)` on `9.99` returns `float`; `type(name)` on `"Rav"` returns `str`. `type()` tells you what kind of value a variable holds.

Not sure what's in a box? Ask Python with **`type()`**:

```
age = 30
print(type(age))      # ->

name = "Rav"
print(type(name))     # ->
```



``type()`` is your little detective — it tells you exactly what kind of value a variable holds. This becomes very handy later when data comes from files or the internet and you're not sure if a value is a number or text. When code misbehaves, checking ``type()`` is often the first clue.

Recap — the 20-second version

- A **variable** is a labelled box: ``name = value`` stores a value under a name.
- ``=`` means "**store this,**" not maths-equals.
- Four everyday types: **int** (whole), **float** (decimal), **str** (text), **bool** (True/False).
- You can **re-fill** a variable — the label stays, the value changes.
- Name things **clearly** (``user_age``, not ``x``); check a type with **type()**.

Next up — Video 3: Strings. You've met text (`str`). Now let's play with it — joining words, pulling out pieces, and the neat f-string trick that makes building messages effortless. See you tomorrow.

